

**VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING**



**WEB APPLICATIONS DEVELOPMENT
IT093IU
LAB 4**

Member List:

- Huỳnh Ngọc Bảo Long – ITITWE23023

Instructor: MSc. Nguyễn Trung Nghĩa

1. Authentication filter

- User accesses any URL (Filter entry point)
 - AuthFilter intercepts EVERY request (/*)
 - Explain:
 - The filter is mapped with: `@WebFilter(urlPatterns = {"/*"})`
 - This means every HTTP request goes through AuthFilter first.
 - The filter casts the request/response:
`HttpServletRequest httpRequest = (HttpServletRequest) request;`
`HttpServletResponse httpResponse = (HttpServletResponse) response;`
 - Then obtains the current path:
`String requestURI = httpRequest.getRequestURI();`
`String contextPath = httpRequest.getContextPath();`
`String path = requestURI.substring(contextPath.length());`
 - Example:
 - ➔ URL = `/student-management-mvc/student?action=list`
 - ➔ contextPath = `/student-management-mvc`
 - ➔ path = `/student?action=list`
- Filter checks if URL is public
 - Public URLs bypass authentication (login, logout, CSS, JS, images).
 - Explain:
 - The method:

```
// Check if this is a public URL
if (isPublicUrl(path)) {
    // Allow access to public URLs
    chain.doFilter(request, response);
    return;
}
```
 - Checks:
 - ➔ `/login`
 - ➔ `/logout`
 - ➔ `.css, .js, .png, .jpg, .jpeg`
 - If the path contains any listed, user is allowed through WITHOUT needing a session ➔ Allows site assets and login page to load correctly

- Filter checks session for logged-in user
 - Filter checks if user attribute exists in session.
 - Explain:
 - Code:


```
HttpSession session = httpRequest.getSession(false);

boolean isLoggedIn = (session != null &&
session.getAttribute("user") != null);
```
 - The logic:
 - ➔ session == null → user not logged in
 - ➔ session exists but no "user" → user not logged in
 - ➔ session exists AND has "user" → user logged in
 - If user is logged in, the filter allows access:


```
chain.doFilter(request, response);
```

 - ➔ Request continues normally to controllers or JSP pages.
- User is NOT logged in → redirect to login
 - Unauthenticated users cannot access protected URLs.
 - Explain:
 - If user is NOT logged in:


```
String loginURL = contextPath + "/login";

httpResponse.sendRedirect(loginURL);
```
 - This ensures:
 - ➔ /student
 - ➔ /dashboard
 - ➔ /student?action=new
 - ➔ /student?action=edit
 - ➔ ANY protected page
 - all redirect to: /login
 - ➔ Only authenticated users can access protected resources.
- Filter lifecycle logging
 - Logs initialization and destruction.
 - Explain:
 - When server starts: `System.out.println("AuthFilter initialized");`
 - When server stops: `System.out.println("AuthFilter destroyed");`
 - These messages confirm that the filter is registered and active.

2. Admin authorization filter

- User accesses a student URL with an action parameter
 - Every request to /student (any action) passes through AdminFilter first.
 - Explain:
 - The filter is mapped to: `@WebFilter(urlPatterns = {"/student"})`
 - This means:
 - `/student?action=list`
 - `/student?action=new`
 - `/student?action=edit&id=5`
 - `/student?action=delete&id=3`
 - All requests go through AdminFilter.
 - The filter gets the action:
`String action = httpRequest.getParameter("action");`
➔ Filter determines what the user is trying to do.
- Filter Checks If the Action Requires Admin Role
 - Only these actions require admin privileges:
 - new
 - insert
 - edit
 - update
 - delete
 - Explain:
 - Method:

```
private boolean isAdminAction(String action) {  
    for (String adminAction : ADMIN_ACTIONS) {  
        if (adminAction.equals(action)) return true;  
    }  
    return false;  
}
```
 - If action matches one of the admin actions:
`isAdminAction(action) == true`
➔ User must be admin to proceed.
 - If not (e.g., list, filter, search):
➔ Normal users are allowed.
- If action requires admin ➔ Validate session & user role
 - Filter checks the session and user object.

- Explain:
 - Gets current session:
`HttpSession session = httpRequest.getSession(false);`
 - If session exists:
`User user = (User) session.getAttribute("user");`
 - Then checks:
`user != null && user.isAdmin()`
 - If true → user is an admin
 - If false → user is not an admin or not logged in
 → Only admins may continue to admin actions.
- Admin access → allow request
 - Admin can manage student data freely.
 - Explain:
 - If `user.isAdmin()` is true: `chain.doFilter(request, response);`
 - This allows the request to continue normally to:
 - Edit student
 - Delete student
 - Insert student
 - Update student
 - Admin receives full access to all management functions.
- Non-admin access attempt → block + show error
 - Non-admin user trying to access admin-only actions gets blocked.
 - Explain:
 - If user is not admin:
`httpResponse.sendRedirect(`
`contextPath + "/student?action=list&error=Access denied. Admin`
`privileges required."`
`);`
 - This adds: `error=Access denied. Admin privileges required.` - to the URL.
 - In `student-list.jsp`, the error message is displayed:

```
<c:if test="${not empty param.error}">
  <div class="alert alert-error">
    ${param.error}
  </div>
</c:if>
```
 - User sees a clear “Access denied” message.

- ➔ Protects the system from unauthorized actions.
- No session ➔ Redirect to login
 - If no session exists, the user is forced to log in again.
 - Explain:

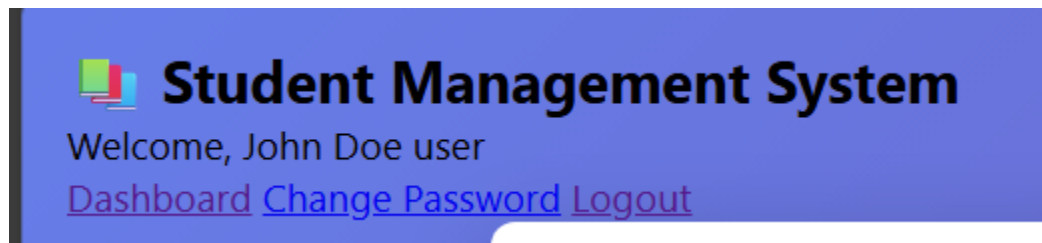

```
if (session == null) {
    httpResponse.sendRedirect(contextPath + "/login");
}
```

 - Prevents anonymous access to admin routes.
- Non-admin actions ➔ Always allowed
 - Actions such as:
 - list
 - search
 - filter
 - sort
 are always permitted.
 - Explain:
 - If isAdminAction(action) is false:


```
chain.doFilter(request, response);
```
 ➔ Regular users can view students but not modify them.

3. Role-Based UI

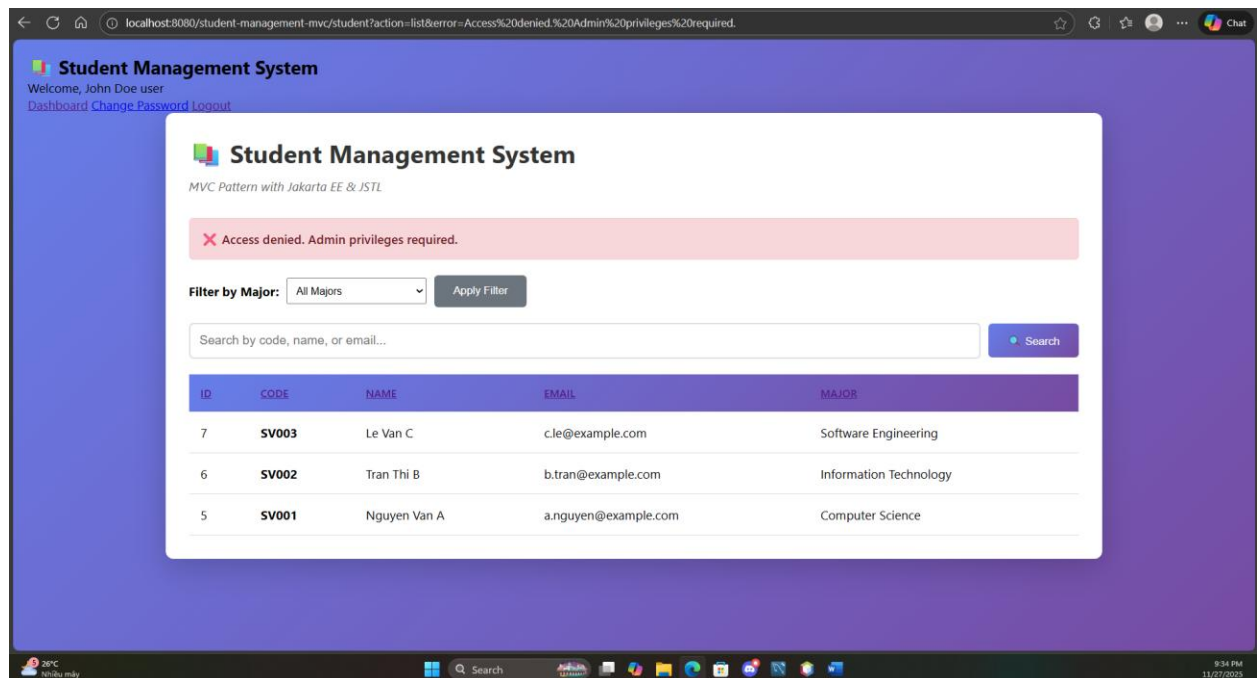
- Navigation bar
 - The navbar displays the logged-in user's full name
 - The user's role appears as a colored badge (e.g., “admin”, “user”)
 - Navbar links: Dashboard, Logout



- Explain:
 - At the top of student-list.jsp, the following is added:


```
<span>Welcome, ${sessionScope.fullName}</span>
<span class="role-badge role-${sessionScope.role}">
    ${sessionScope.role}
</span>
```

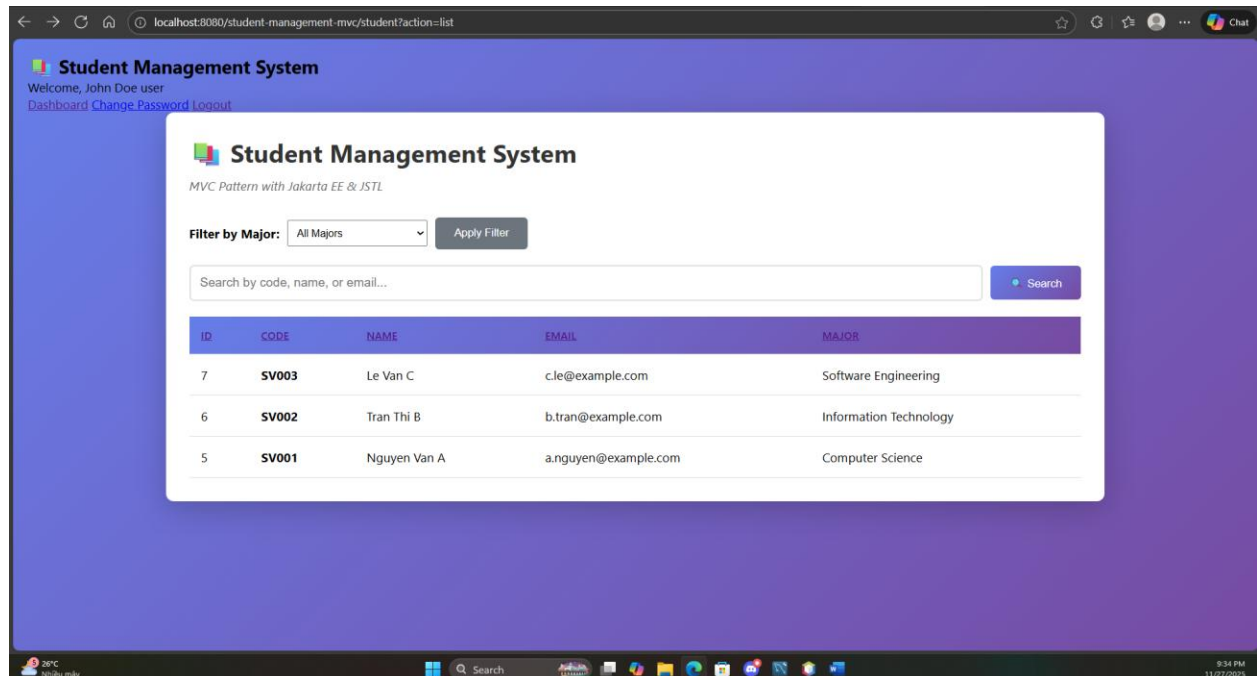
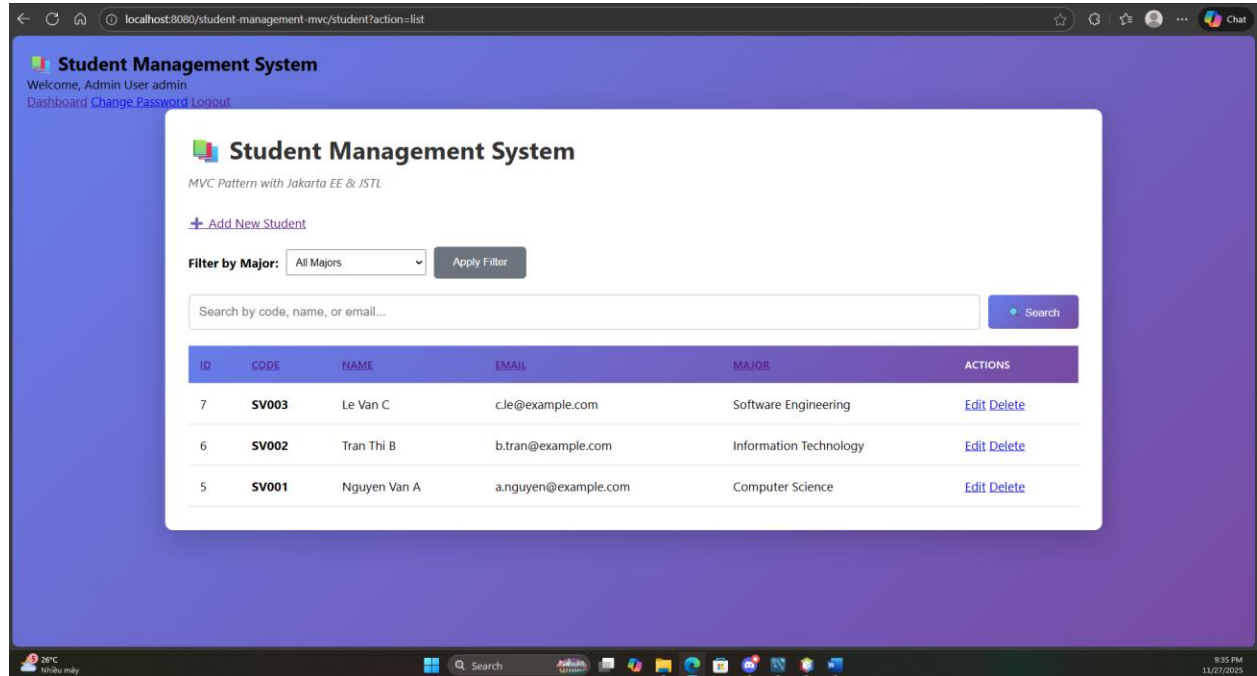
- This uses session attributes set at login:
`${sessionScope.fullName}` shows the user's name
`${sessionScope.role}` shows the role (admin/user)
- Because these values come from the session, they update dynamically based on the logged-in user.
 ➔ This provides user context and shows role visually on every page.
- Display error message from AdminFilter (Unauthorized access)
 - If a non-admin tries to access an admin-only URL (e.g., `/student?action=new`),
 - AdminFilter sends an error using: `?error=Access denied`
 - The page displays the message.



- Explain:
 - In student-list.jsp:


```
<c:if test="${not empty param.error}">
  <div class="alert alert-error">
    ${param.error}
  </div>
</c:if>
```
 - This checks the URL for: `?error=some message`
 - If present, it displays the error box.
 ➔ This helps notify users when they attempt unauthorized actions.

- “Add new student” button - admin only
 - Admins see the “+ Add new student” button
 - Regular users do NOT see the button



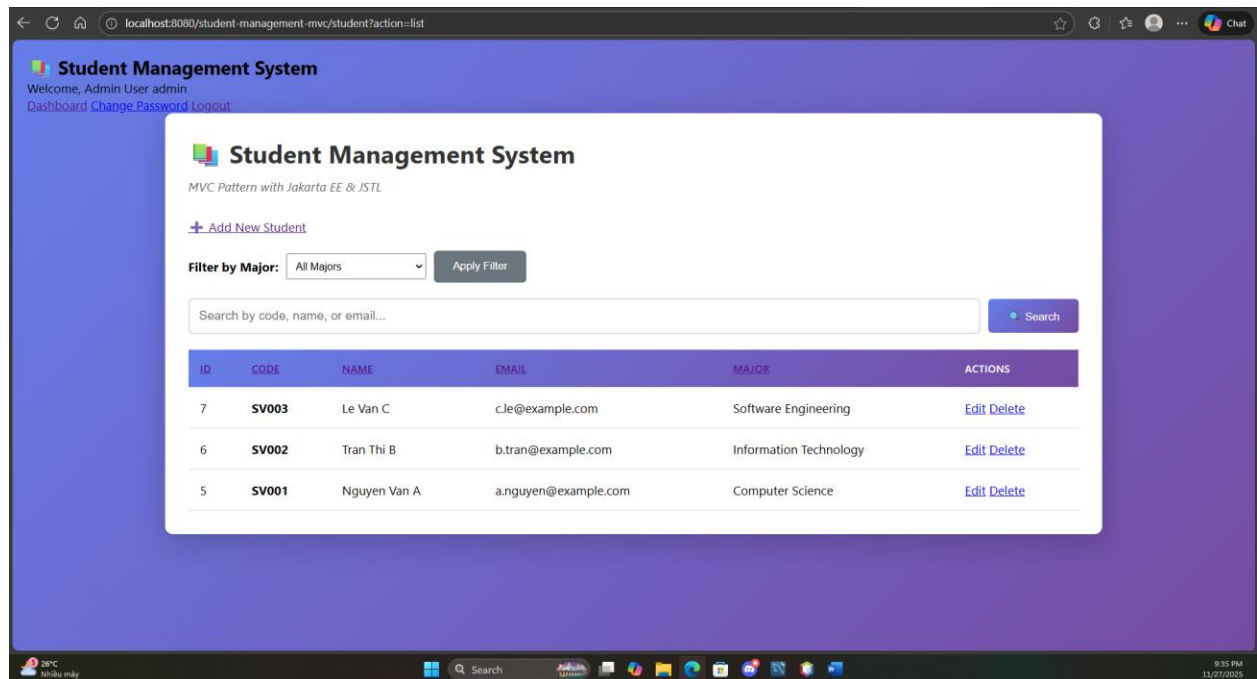
- Explain:

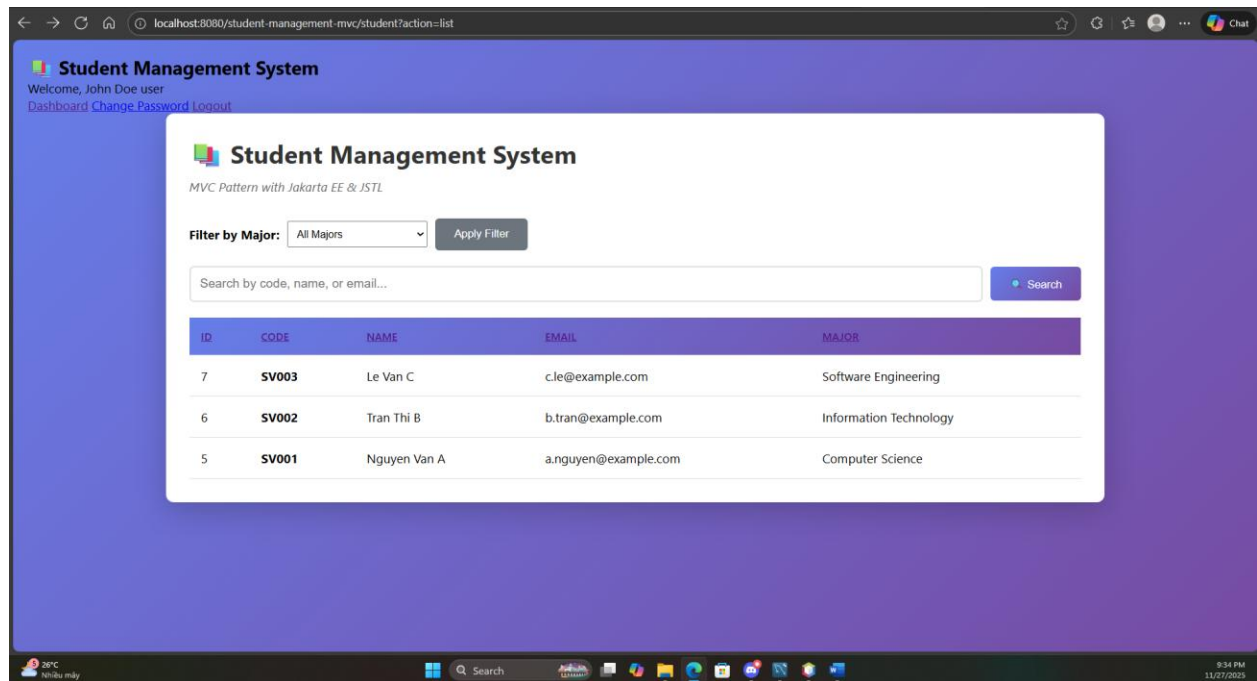

```
<c:if test="${sessionScope.role eq 'admin'}">
  <a href="student?action=new">+ Add new student</a>
```


</c:if>

- This checks: session role == "admin"
- If true → show the button
- If false → hide it completely
- ➔ UI cleanly restricts access based on role.
- ➔ Regular users won't even know the feature exists.

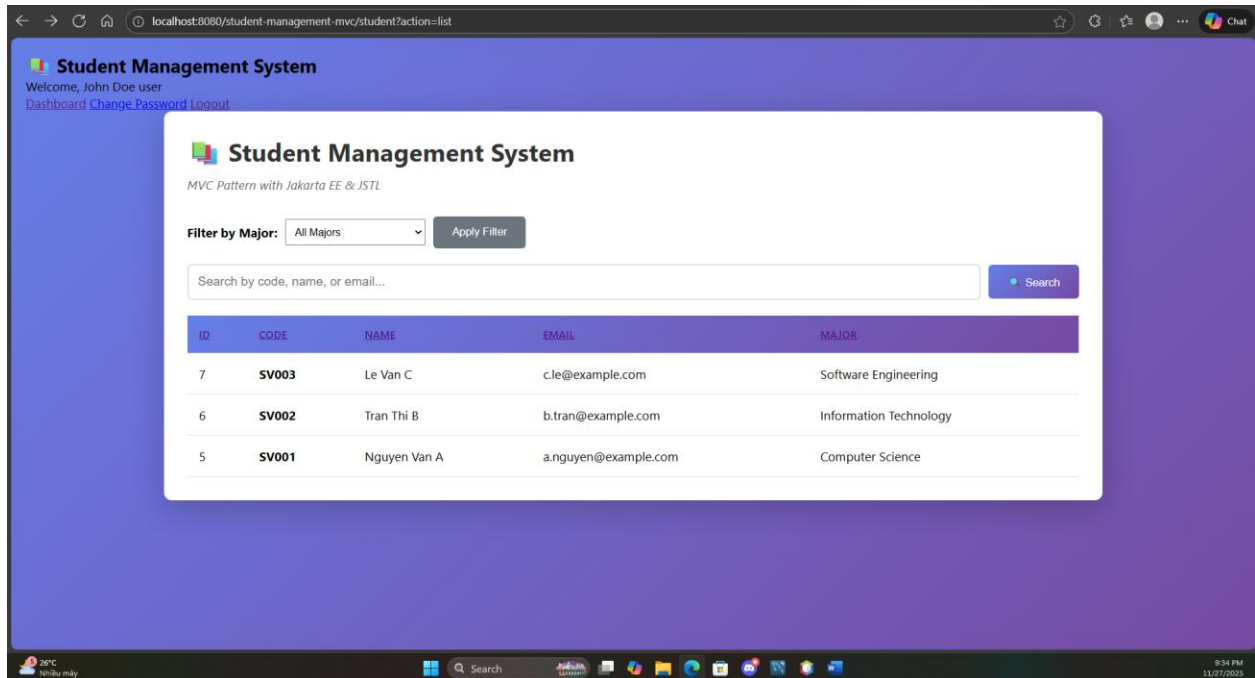
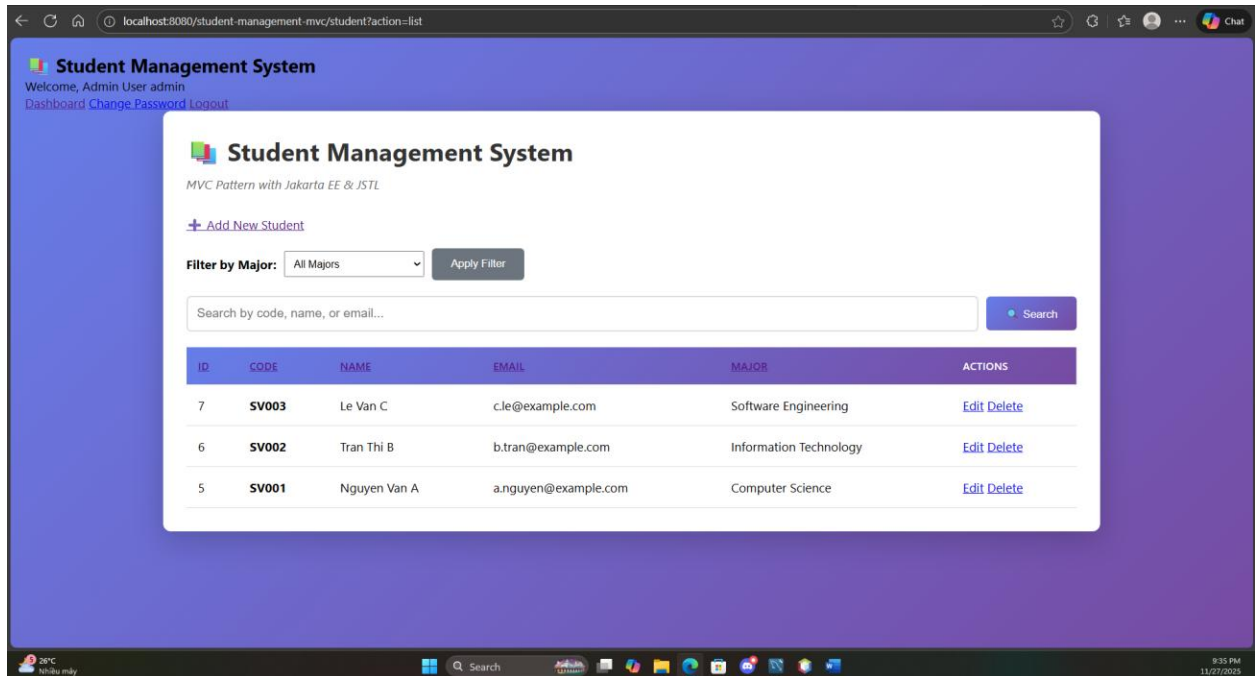
- Table header - show "Actions" column only for admins
 - Admin sees an "Actions" column
 - Regular users do not





- Explain:
 - Inside <thead>:


```
<c:if test='${sessionScope.role eq 'admin'}'>
    <th>Actions</th>
</c:if>
```
 - Because table headers must match table rows, hiding this column prevents layout misalignment.
 - ➔ Only admins have an extra column for editing/deleting student records.
- Table row Actions - Edit/Delete buttons admin only
 - Admins see Edit/Delete buttons beside each student
 - Regular users do not see these buttons at all



- Explain:
 - Inside each `<tr>` row:

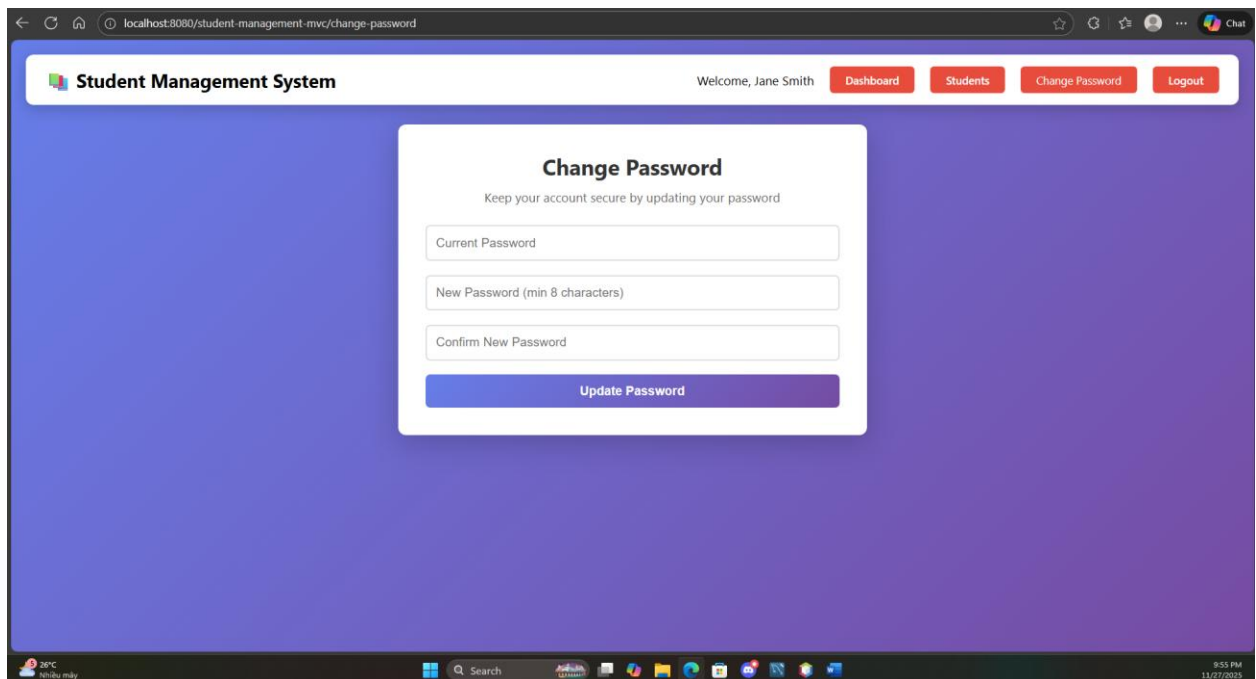

```
<c:if test="\${sessionScope.role eq 'admin'}">
    <td>
      <a href="student?action=edit&id=\${student.id}">Edit</a>
      <a
        href="student?action=delete&id=\${student.id}">Delete</a>
```

```
</td>  
</c:if>
```

- This ensures:
 - Buttons are rendered ONLY when role = admin
 - Regular users cannot modify data
 - Combined with AdminFilter, this protects both UI and backend security
- ➔ UI-based access control + backend filter reinforcement.

4. Change password

- User opens change password page (Access: /change-password via navbar link)
 - The change-password.jsp form is displayed
 - User sees fields:
 - Current Password
 - New Password
 - Confirm Password



- Explain:
 - When the user accesses: GET /change-password
 - The controller runs:
`request.getRequestDispatcher("views/change-password.jsp").forward(request, response);`

- This simply loads the JSP form.
 - No validation yet
 - User can enter password details
- User submits the form (POST) (Submit button → POST /change-password)
 - Server receives currentPassword, newPassword, confirmPassword
 - The controller begins validation

The image shows a web form titled "Change Password" with the subtitle "Keep your account secure by updating your password". The form is set against a white background with a purple border. It contains three input fields, each with a light gray border and rounded corners. The first two fields are for "currentPassword" and "newPassword", each containing a series of dots representing masked text. The third field is for "confirmPassword", also containing dots, and it includes a small eye icon on the right side to toggle password visibility. Below the input fields is a wide, rounded rectangular button with a blue-to-purple gradient, labeled "Update Password" in white text.

- Explain:
 - The controller reads the form fields:


```
String currentPassword =
request.getParameter("currentPassword");
String newPassword = request.getParameter("newPassword");
String confirmPassword =
request.getParameter("confirmPassword");
```
 - The user object is obtained from session:


```
User currentUser = (User) session.getAttribute("user");
```
 - If session is missing (not logged in), user is redirected to login.
 - ➔ Only authenticated users can change password

- Validate current password
 - If current password does NOT match the database hash → show error

The image shows a 'Change Password' form within a purple-bordered container. The form has a white background and rounded corners. At the top, the title 'Change Password' is in bold black text, followed by the subtitle 'Keep your account secure by updating your password' in a smaller, lighter font. Below this is a red error message box with a red 'X' icon and the text 'Current password is incorrect.'. Underneath the error message are three input fields: 'Current Password', 'New Password (min 8 characters)', and 'Confirm New Password'. At the bottom of the form is a blue button with the text 'Update Password'.

- Explain:
 - BCrypt verifies the input password:

```
if (!BCrypt.checkpw(currentPassword, currentUser.getPassword())) {  
    request.setAttribute("error", "Current password is incorrect.");  
    return;  
}
```
 - If invalid → JSP displays: **X** Current password is incorrect.
 - Ensures the user is the legitimate account owner
- Validate new password requirements
 - New password must have at least 8 characters
 - Confirm Password must match New Password

Change Password

Keep your account secure by updating your password

✖ New password must be at least 8 characters.

Update Password

Change Password

Keep your account secure by updating your password

✖ New passwords do not match.

Current Password

New Password (min 8 characters)

Confirm New Password

Update Password

- Explain:
 - Check minimum length:

```
if (newPassword.length() < 8) {  
    set error: "New password must be at least 8 characters."  
}
```
 - Check both fields match:

```
if (!newPassword.equals(confirmPassword)) {  
    set error: "New passwords do not match."  
}
```
 - If any validation fails, the page reloads with an error message.
 - ➔ Ensures password meets basic security rules
 - ➔ Prevents typos or weak passwords
- Hash the new password (BCrypt)
 - If all validation passes ➔ new password is securely hashed
 - Explain:
 - The controller hashes the password:


```
String hashedPassword = BCrypt.hashpw(newPassword,  
BCrypt.gensalt());
```

- This produces a 60-character encrypted string, impossible to reverse.

➔ Protects password integrity

➔ Critical for secure authentication

- Update password in database

- UserDao updates the user's stored password

- Explain:

```
boolean updated = userDao.updatePassword(currentUser.getId(),  
hashedPassword);
```

- DAO executes: UPDATE users SET password = ? WHERE id = ?

- If successful ➔ Password updated in database

- Also updates the user object in session:

```
currentUser.setPassword(hashedPassword);
```

➔ Session and database stay synchronized

- Show success or error message

- JSP displays success message or failure

Change Password

Keep your account secure by updating your password

✓ Password updated successfully!

Current Password

New Password (min 8 characters)

Confirm New Password

Update Password

- Explain:
 - If update is successful:
`request.setAttribute("message", "Password updated successfully!");`
 - Else:
`request.setAttribute("error", "Failed to update password. Try again.");`
 - The JSP checks:
`<c:if test="${not empty message}">`
 show green success box
`</c:if>`

`<c:if test="${not empty error}">`
 show red error box
`</c:if>`
- ➔ User gets clear feedback about the update result